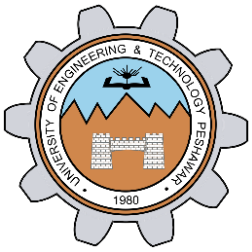
Lab # 06

Linked List Using Pointers



Spring 2026

Data Structures And Algorithms Lab

Submitted by: **M Mohsin Khan**

Registration No.: **24PWCSE2382**

Class Section: **C**

Submitted to:

Engr. Abdullah Hamid

June 20th , 2026

Department of Computer Systems Engineering

University of Engineering and Technology, Peshawar

Task 01:

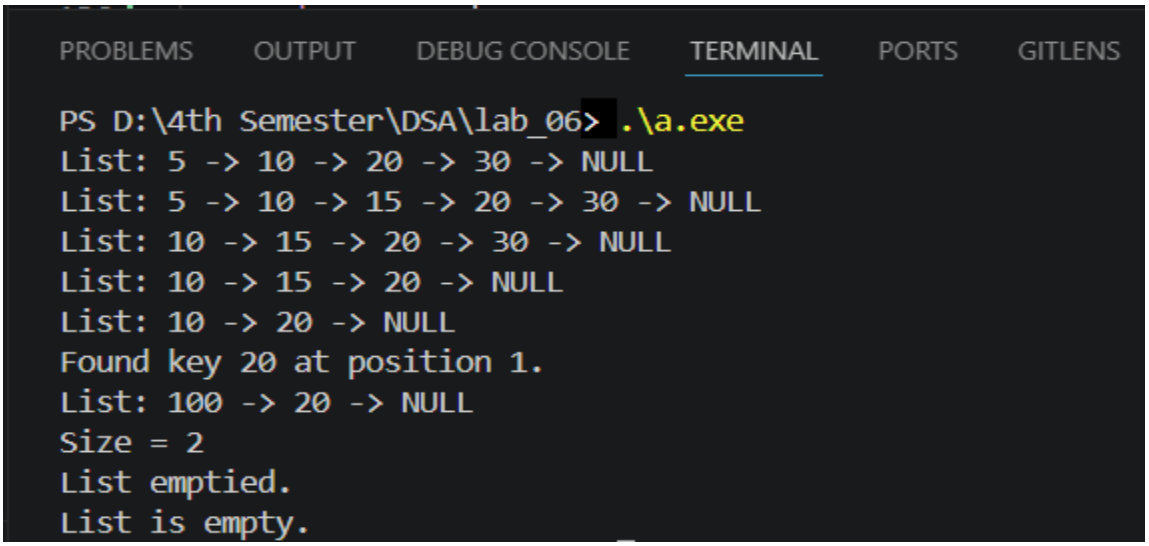
```cpp
#include <iostream>
using namespace std;
struct Node
{
    int data;
    Node *next;
};

Node *createNode(int data)
{
    Node *newNode = new Node;
    if (!newNode)
    {
        cout << "Memory allocation
failed.\n";
        exit(1);
    }

    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
Node *getNodeAt(Node *head, int index)
{
    while (head && index--)
    {
        head = head->next;
    }
    return head;
}

void insertNode(Node **head, int data, int
position)
{
    if (position == 0)
    {
        Node *newNode = createNode(data);
        newNode->next = *head;
        *head = newNode;
    }
    else
    {
        Node *prev = getNodeAt(*head,
position - 1);
        Node *newNode = createNode(data);
        newNode->next = prev->next; //
points the newNode to the next/after node
if we are inserting in the middle of the
list
        prev->next = newNode;       //
detach the previous node and attach to the
new one
    }
}

void deleteAt(Node **head, int position)
{

    // inefficient approach and not
handling some invalide cases
    // Node *prev = getNodeAt(*head,
position - 1);
    // Node *currentNode = getNodeAt(*head,
position);
    // Node *nextNode = getNodeAt(*head,
position + 1);
    // prev->next = nextNode;
    // delete currentNode;

    if (position == 0)
    {
        Node *temp = *head;
        *head = (*head)->next;
        delete temp;
        return;
    }

    Node *prev = getNodeAt(*head, position
- 1);

    if (prev == NULL || prev->next == NULL)
    {
        return;
    }
    Node *currentNode = prev->next;
    prev->next = currentNode->next;
    delete currentNode;
}
void Search(Node *head, int key)
{
    int pos = 0;
    while (head)
    {
        if (head->data == key)
        {
```

```
            cout << "Found key " << key <<
" at position " << pos <<

                ".\n";

            return;
        }
        head = head->next;
        pos++;
    }
    cout << "Key " << key << " not
found.\n";
}
void EmptyList(Node **head)
{
    Node *temp;
    while (*head)
    {
        temp = *head;
        *head = (*head)->next;
        delete temp;
    }
    cout << "List emptied.\n";
}
void Print(Node *head)
{
    if (!head)
    {
        cout << "List is empty.\n";
        return;
    }
    cout << "List: ";
    while (head)
    {
        cout << head->data << " -> ";
        head = head->next;
    }
    cout << "NULL\n";
}
int Size(Node *head)
{
    int count = 0;

    while (head)
    {
        count++;
        head = head->next;
    }

    return count;
}
void Update(Node *head, int position, int
newValue)
{
    Node *target = getNodeAt(head,
position);

    if (target == NULL)
    {
        cout << "Invalid position.\n";
        return;
    }

    target->data = newValue;
}
void deleteFromEnd(Node **head)
{
    if (*head == NULL)
        return;

    if ((*head)->next == NULL)
    {
        delete *head;
        *head = NULL;
        return;
    }

    Node *temp = *head;

    while (temp->next->next)
    {
        temp = temp->next;
    }

    delete temp->next;
    temp->next = NULL;
}
void deleteFromStart(Node **head)
{
    if (*head == NULL)
        return;

    Node *temp = *head;
    *head = (*head)->next;

    delete temp;
}
void insertAtEnd(Node **head, int data)
{
    Node *newNode = createNode(data);
```

```
    if (*head == NULL)
    {
        *head = newNode;
        return;
    }

    Node *temp = *head;

    while (temp->next)
    {
        temp = temp->next;
    }

    temp->next = newNode;
}
void insertAtStart(Node **head, int data)
{
    insertNode(head, data, 0);
}
int main()
{
    Node *head = NULL;

    insertAtStart(&head, 10);
    insertAtStart(&head, 5);

    insertAtEnd(&head, 20);
    insertAtEnd(&head, 30);

    Print(head);

    insertNode(&head, 15, 2);
```

```
    Print(head);

    deleteFromStart(&head);

    Print(head);

    deleteFromEnd(&head);

    Print(head);

    deleteAt(&head, 1);

    Print(head);

    Search(head, 20);

    Update(head, 0, 100);

    Print(head);

    cout << "Size = "
         << Size(head)
         << endl;

    EmptyList(&head);

    Print(head);

    return 0;
}
```

Terminal Output:



RUBRICS (LABS/ PROJECTS):

Criteria & Point Assigned	**Outstanding 4**	**Acceptable 3**	**Considerable 2**	**Below Expectations 1**	**Score**
Valuing related to work, including responsibility, and professionalism.	Demonstrates **excellent** responsibility and professionalism in task managment. Shows **strong commitment to** quality work, meeting the deadline, and comprehensive, well-organized **effective documentation** following all guidelines with exemplary attention to detail.	Demonstrates **some** responsibility and professionalism. Displays **moderate commitment to** quality work, meets the deadline and shows **sufficient thoroughness** in documentation, adhering to guidelines, with some room for enhancement.	Demonstrates **limited effort** in responsibility and professionalism. Misses the deadline with **notable inconsistency** in quality work and shows **insufficient** thoroughness or clarity in documentation, necessitating improvements.	Demonstrates a **lack** of responsibility and professionalism. Misses the deadline and **lacks** quality work. Documentation is incomplete, disorganized, or unclear.	
Ability to apply techniques, methods, and procedures effectively.	Executes lab tasks **with exceptional skill** to design/develop solution and **shows proficiency** in lab techniques and procedures.	Executes lab tasks **with adequate skill** to design/develop solution and **shows proficiency** in lab techniques and procedures.	Executes lab tasks **with minimal skill** to design/develop solution and **shows modeate proficiency** in lab techniques and procedures.	Executes lab tasks **with limited skill** to design/develop solution and **lacks proficiency** in lab techniques and procedures.	

Demonstrate understanding and relevant Knowledge.	Demonstrates **exceptional** understanding of concepts with **clear explanation** of investigation and analysis of the objectives and outcomes.	Demonstrates a **good** understanding of concepts with **some explanations** of investigation and analysis that align with the lab's objectives and outcomes.	Demonstrates a **basic** understanding of key concepts but **lacks clarity or depth in explaining** the investigation and analysis, with some inconsistencies.	Demonstrates **limited** understanding of concepts with **unclear explanations** of investigation and analysis that do not align with the lab's objectives and outcomes.	